CHANDIGARH UNIVERSITY

PROJECT REPORT

STUDENT REGISTRATION FORM

Design, Implementation and Testing of a Python Tkinter Application

SUBMITTED BY
RIWANG CHOKI TSHERING
UID: 24BC10525
Bachelor of Computer Applications (BCA)
Subject: Python Programming
Academic Year: 2026

A practical project using Python and the Tkinter GUI library

CERTIFICATE

This is to certify that the project report entitled "Student Registration Form – A Python Tkinter Application" is a bona fide record of work carried out by **Riwang Choki Tshering** (UID: 24BC10525) in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) at Chandigarh University.

The work embodied in this report was carried out under supervision and, to the best of my knowledge, has not been submitted to any other university or institution for the award of any degree or diploma.

Project Guide: _____
Signature: _____
Date: _____

STUDENT DECLARATION

I hereby declare that this project report titled "Student Registration Form – A Python Tkinter Application" has been prepared for academic learning and submission in the subject of Python Programming. The design, source code, tables, diagrams and test results presented in this report are the outcome of my own study and implementation.

The sample records shown in the figures and the test table are hypothetical and are used only for educational demonstration. Published documentation and references used during the work are acknowledged in the references.

Student Name: Riwang Choki Tshering
UID: 24BC10525
Programme: Bachelor of Computer Applications (BCA)
Subject: Python Programming

Signature: _____

Date: _____

PROJECT INFORMATION

Item	Description
Project title	Student Registration Form – A Python Tkinter Application
Core area	Python Programming and Graphical User Interface (GUI) Development
Tools used	Python 3, tkinter, tkinter.messagebox, tkinter.ttk (standard library only)
Data storage	In-memory list of Python dictionaries (one dictionary per student)
Input attributes	Name, Roll Number, Email, Gender, Course, Hobbies
Validation rules	No empty fields; email must contain "@" and "."; roll number must be unique
Expected learning outcome	Understand event-driven GUI programming, input validation and basic data handling

ACKNOWLEDGEMENT

I express my sincere gratitude to Chandigarh University for providing the academic environment and resources needed to complete this project. I am thankful to my project guide and the faculty members for their guidance, valuable suggestions and encouragement, which helped me understand the practical side of programming.

I also thank my friends and classmates for testing the application and sharing useful ideas, and my family for their constant support and motivation.

ABSTRACT

Manual, paper-based student registration is slow, error-prone and difficult to search. This project, Student Registration Form, is a desktop application developed in Python using the Tkinter GUI toolkit. It lets a user register students through a graphical form and view all registered records in a table.

The form collects the student's name, roll number, email address, gender, course and hobbies using text entries, radio buttons, a drop-down menu and check buttons. Before a record is saved, the program checks that no field is empty, that the email address has a valid basic format and that the roll number has not been used before. Valid records are stored as dictionaries in a list and can be displayed in a separate window using a Treeview table with a scroll bar and a total count.

The report explains the objectives, requirements, design, methodology, implementation and testing of the application, and lists its limitations and future scope. The central finding is that a small, dependency-free Tkinter program can provide a usable registration workflow with basic validation, while permanent storage and stronger validation remain the main areas for improvement.

TABLE OF CONTENTS

Section	Contents	Page
Preliminary pages	Certificate, declaration, project information, acknowledgement and abstract	2
1	Introduction	5
2	Project objectives and requirements	5
3	System design	6
4	Methodology	8
5	Implementation	9
6	Testing	10
7	Limitations and future scope	11
8	Conclusion	12
References	Bibliography	12
Appendix A	Key terms	12

LIST OF FIGURES

Figure	Title
1	Add Student: validation and storage workflow
2	Layout of the main registration window
3	Layout of the records window

LIST OF TABLES

Table	Title
1	Functional requirements
2	Hardware and software requirements
3	Structure of a student record
4	Tkinter widgets used
5	Input fields and validation rules
6	Development phases
7	Functions in the program
8	Test cases and observed results

1. INTRODUCTION

1.1 Background

Registering students is one of the most basic administrative tasks in any school, college or training centre. When it is done on paper or in unstructured spreadsheets, the process leads to missing data, duplicate entries and wasted time. A small GUI-based tool that checks data at the moment of entry can remove many of these problems.

Python is a popular language for such tools because of its readable syntax and rich standard library. Tkinter, the standard GUI package bundled with Python, offers all the widgets needed to build a registration form without installing anything extra. This project uses Tkinter to build a colour-themed registration form with an integrated record viewer.

1.2 Problem statement

To design and develop a desktop application that accepts student details through a user-friendly form, validates the entered data, prevents duplicate roll numbers and shows all registered students in a tabular view.

1.3 Scope

The project covers data entry, validation, in-memory storage and display of student records during one run of the program. It is a learning project and a base that can later be extended with permanent storage, editing and searching. It is not intended to replace a real student-information system.

2. PROJECT OBJECTIVES AND REQUIREMENTS

2.1 Objectives

- Build a graphical student registration form using Python and Tkinter.
- Use different input widgets (Entry, Radiobutton, OptionMenu, Checkbutton) in a single application.
- Validate user input: empty fields, email format and duplicate roll numbers.
- Store student records in a structured form using a list of dictionaries.
- Display all registered students in a table with a scroll bar and a total count.
- Give clear feedback through message boxes and allow the form to be cleared quickly.

2.2 Functional requirements

ID	Requirement
FR1	Accept the student name, roll number and email address as text input.
FR2	Let the user choose gender (Male, Female, Other) using radio buttons.
FR3	Let the user select a course (BCA, BBA, B.Tech, MCA, MBA, B.Sc) from a drop-down list.
FR4	Let the user select zero or more hobbies (Reading, Music, Sports, Travelling).
FR5	Reject a submission if a text field is empty, the email is invalid or the roll number already exists.
FR6	Store each valid record and show a success message with the total number of students.
FR7	Display all records in a table in a separate window.
FR8	Provide a button that clears the form.

2.3 Non-functional requirements

- **Usability:** a simple, clearly labelled form with a consistent colour theme.
- **Portability:** runs on Windows, Linux and macOS wherever Python with Tkinter is available.
- **Performance:** instant response for the expected number of records.
- **Maintainability:** separate, commented functions for clearing, adding and displaying records.

2.4 Hardware and software requirements

Item	Specification
Processor	Any modern dual-core processor
RAM	2 GB or more
Storage	Less than 50 MB free space
Operating system	Windows 10/11, Linux or macOS
Language	Python 3.x
Libraries	tkinter, tkinter.messagebox, tkinter.ttk (Python standard library)
Editor / IDE	IDLE, VS Code, PyCharm or any text editor

3. SYSTEM DESIGN

3.1 Architecture

The application follows a single-window, event-driven design. The main window holds the registration form. A button click triggers a callback function that reads the widget values, validates them, updates the data store and gives feedback. A second window (Toplevel) is created on demand to show the records.

3.2 Data design

All records are kept in a global list named `student_list`. Each student is stored as a dictionary with the keys shown below; roll numbers are stored as text.

Key	Type	Description
Name	str	Full name of the student
Roll Number	str	Unique identifier (duplicates are rejected)
Email	str	Email address (basic format check)
Gender	str	Male, Female or Other
Course	str	Selected course
Hobbies	str	Comma-separated hobbies, or “None”

3.3 User interface design

The main window is 550 × 700 pixels, has a fixed size and uses a light blue background with a white form panel. The fields are placed with the grid layout manager inside the panel and the three action buttons are stacked below it. The records window uses a themed Treeview with a vertical scroll bar.

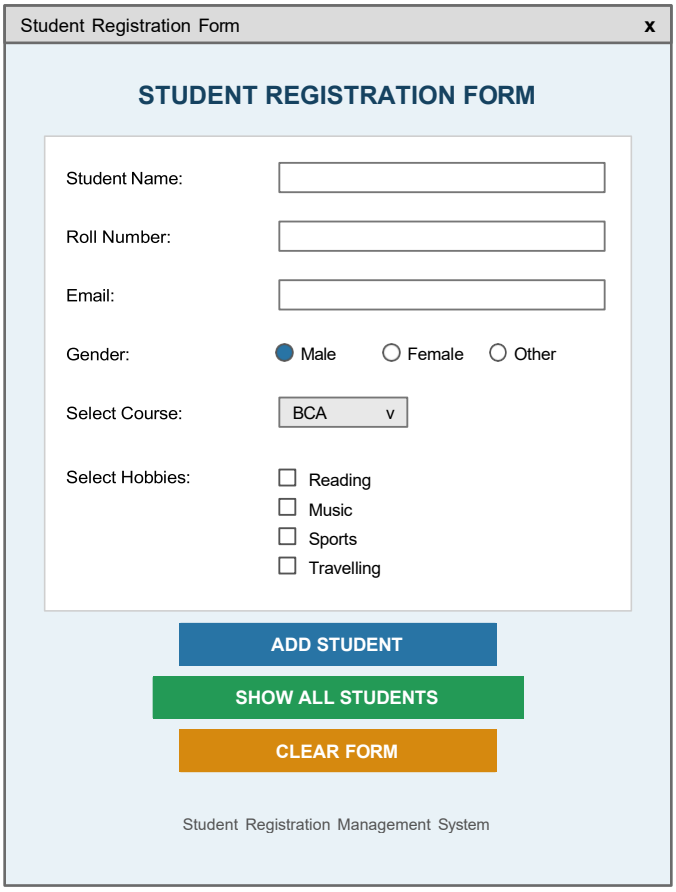


Figure 2. Layout of the main registration window (schematic drawing of the interface built by the program).

Widget	Used for
Label	Headings and field captions
Entry	Name, roll number and email
Radiobutton	Gender (single choice)
OptionMenu	Course selection
Checkbutton	Hobbies (multiple choice)
Button	ADD STUDENT, SHOW ALL STUDENTS, CLEAR FORM
Toplevel, Treeview, Scrollbar	Table of all registered students
messagebox	Warnings and success messages

Field	Widget	Validation / default
Student Name	Entry	Required; leading and trailing spaces removed
Roll Number	Entry	Required; must not already exist in the list
Email	Entry	Required; must contain "@" and "."
Gender	Radiobutton	Default: Male
Course	OptionMenu	Default: BCA
Hobbies	Checkbutton (4)	Optional; stored as "None" if nothing is ticked

4. METHODOLOGY

The project was developed incrementally: the form was built first, followed by the clear, add and display functions, each tested before the next was added.

Phase	Activity
1. Requirement analysis	Identified the fields to collect and the validation rules.
2. Design	Sketched the form layout and chose the data structure.
3. Coding	Built the widgets, then the clear, add and display functions one at a time.
4. Testing	Tested each function with valid and invalid inputs.
5. Refinement	Improved layout, colours and messages based on the test results.

4.1 Flow of the Add Student operation

When ADD STUDENT is pressed, the program stops at the first failed check and shows a warning; a record is stored only if all three checks pass.

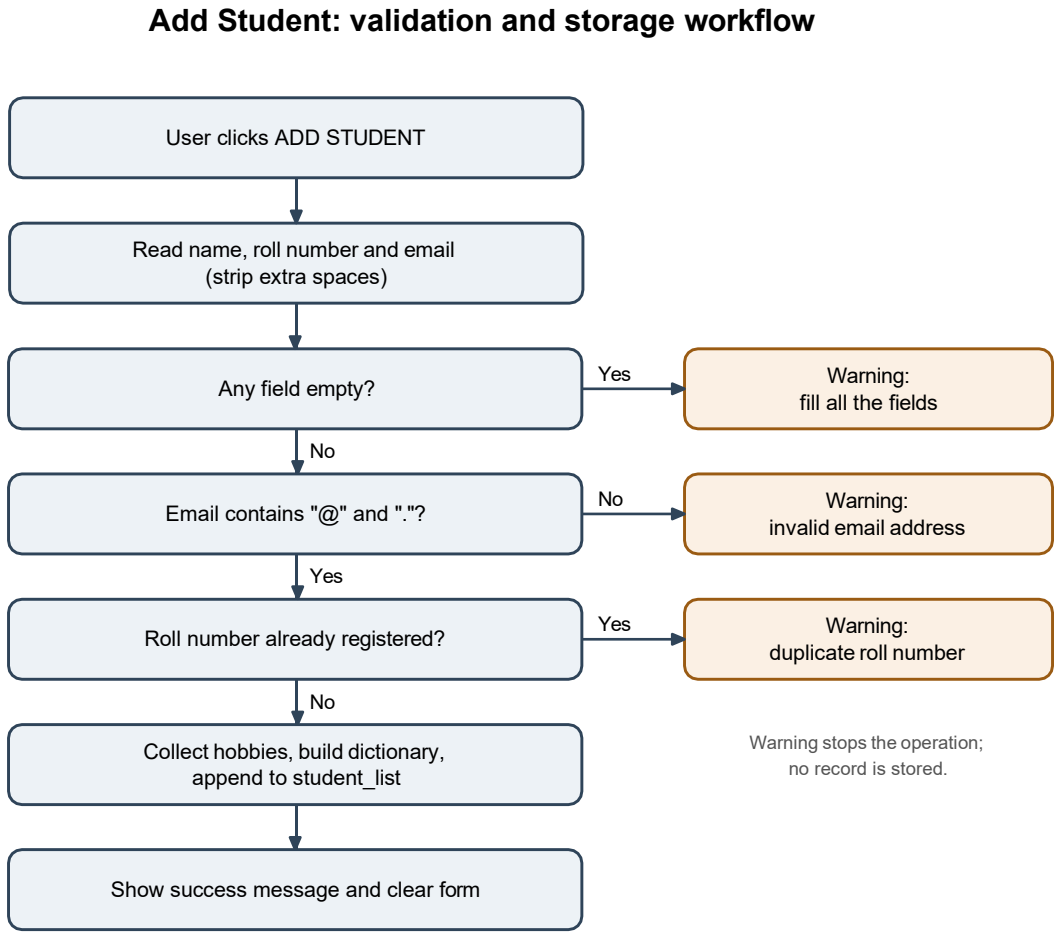


Figure 1. Workflow of the Add Student operation. Each failed check shows a warning and stops without storing a record.

5. IMPLEMENTATION

The complete source code is given in Appendix B. The program is written in a procedural style using callback functions; the important parts are explained below.

5.1 Main window and form

A `Tk()` root window is created with a fixed size and background colour. A heading label and a white `Frame` hold the form fields. `StringVar` objects are bound to the radio buttons and the drop-down menu, and `BooleanVar` objects are bound to the four check buttons, so their current values can be read easily.

5.2 Functions

Function	Purpose
`clear_form()`	Empties the three entry boxes, resets gender to Male and course to BCA, and unticks all hobbies.
`add_student()`	Reads and trims the entries, applies the three validation checks, collects hobbies, builds the dictionary, appends it to the list, shows a confirmation and clears the form.
`show_all_students()`	Shows a warning if the list is empty; otherwise opens a Toplevel window with a six-column Treeview, a scroll bar, one row per student and a total count.

5.3 Validation logic

Validation happens in a fixed order: (1) any of name, roll number or email empty; (2) email lacking "@" or "."; (3) a roll number equal to one already stored. Hobbies are gathered afterwards, and if none are ticked the value "None" is stored so that the table never shows a blank cell.

5.4 Records window

The records window builds its table from the column names Name, Roll Number, Email, Gender, Course and Hobbies and inserts each dictionary as one row. The window shows the count at the time it was opened.

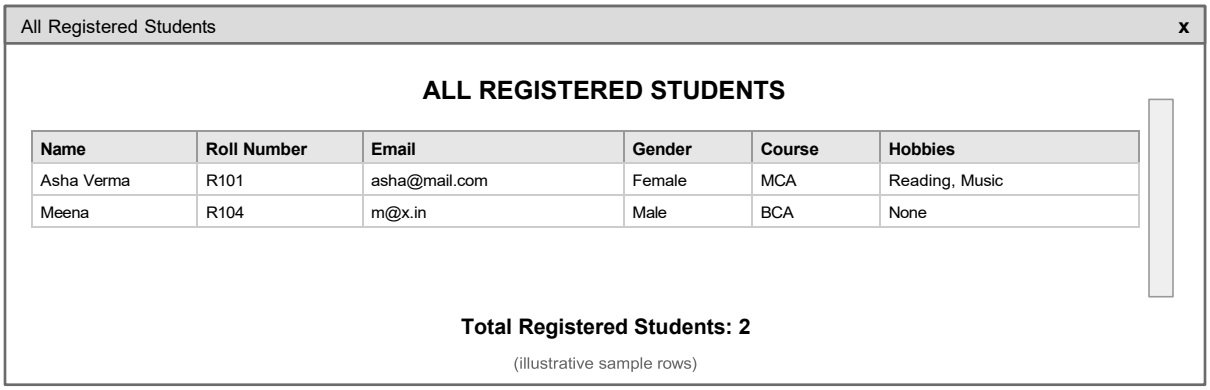


Figure 3. Layout of the records window opened by SHOW ALL STUDENTS (sample rows are illustrative).

5.5 Buttons and main loop

The three buttons are connected to the functions with the `command` option. Finally, `root.mainloop()` starts the event loop that waits for user actions.

6. TESTING

Black-box testing was used to check that the program behaves correctly for valid and invalid input. Each case was run by setting the form values and calling the program's functions, then recording the message box shown and the resulting contents of the student list. The sample data below is hypothetical.

No.	Test case	Input	Expected result	Observed
1	Valid registration	Asha Verma, R101, asha@mail.com, Female, MCA, Reading + Music	Success message; total becomes 1; form cleared	Pass: "Total Students: 1"; fields cleared, course reset to BCA

No.	Test case	Input	Expected result	Observed
2	Empty field	Name, roll number or email left blank (each tried)	Warning: "Please fill all the fields!"	Pass: warning shown; nothing stored
3	Invalid email	abc.com; abc@com; plain	Warning: "Enter a valid email address!"	Pass: warning for all three
4	Duplicate roll number	R101 entered again	Warning: "This roll number is already registered!"	Pass: warning shown; count unchanged
5	No hobby selected	All check boxes unticked	Saved with Hobbies = "None"	Pass: stored as "None"
6	All hobbies, other options	Other, B.Tech, all four hobbies	Hobbies stored as a comma-separated list	Pass: "Reading, Music, Sports, Travelling"
7	Extra spaces	" Tara ", " R106 ", " t@x.in "	Values stored without surrounding spaces	Pass: "Tara", "R106", "t@x.in"
8	Show all, empty list	Press SHOW ALL before adding anyone	Warning: "No students have been registered yet!"	Pass: warning shown; no window
9	Show all with data	Four students registered	Table with one row per student and total count	Pass: six columns, four rows
10	Clear form	All fields filled, then CLEAR FORM	All fields return to defaults	Pass: text empty, Male, BCA, hobbies unticked

Result: all ten test cases behaved as expected. The tests cover the program logic; the on-screen appearance of the windows should be confirmed by running the program and capturing screenshots for the final submission.

7. LIMITATIONS AND FUTURE SCOPE

7.1 Limitations

- Records are held only in memory and are lost when the program is closed.
- Email validation is basic: it only checks for "@" and ".", so an address such as a.b@c is accepted.
- Roll numbers are compared as exact text, so R101 and r101 count as different roll numbers.
- There is no option to edit or delete a record once it has been added.
- The records window is a snapshot: it does not refresh when more students are added, and pressing SHOW ALL again opens another window.

7.2 Future scope

- Save records permanently in a CSV/JSON file or a database such as SQLite or MySQL.
- Add edit, delete, search and sort options.
- Use regular expressions for stronger email and roll-number validation, and ignore letter case when checking duplicates.
- Add an administrator login and export of records to Excel or PDF.

8. CONCLUSION

The Student Registration Form project meets its objectives. It provides a simple, user-friendly way to enter student details, checks the data for common mistakes, prevents duplicate roll numbers and shows all registered students in a table. Through this project I gained practical experience with Tkinter widgets, layout management, event handling, lists and dictionaries, and input validation.

Although the present version stores data only temporarily, its modular structure makes it easy to extend with permanent storage and more features. The project is a good foundation for larger information-management applications.

REFERENCES

Python Software Foundation. tkinter — Python interface to Tcl/Tk. Python 3 Documentation. https://docs.python.org/3/library/tkinter.html

Python Software Foundation. tkinter.ttk — Tk themed widgets. Python 3 Documentation. https://docs.python.org/3/library/tkinter.ttk.html

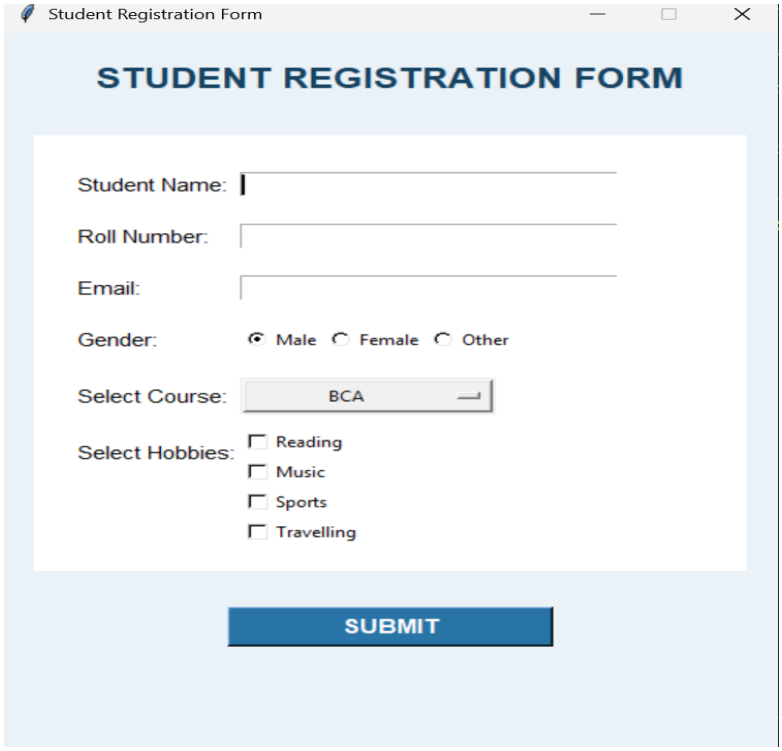
Python Software Foundation. tkinter.messagebox — Tkinter message prompts. Python 3 Documentation. https://docs.python.org/3/library/tkinter.messagebox.html

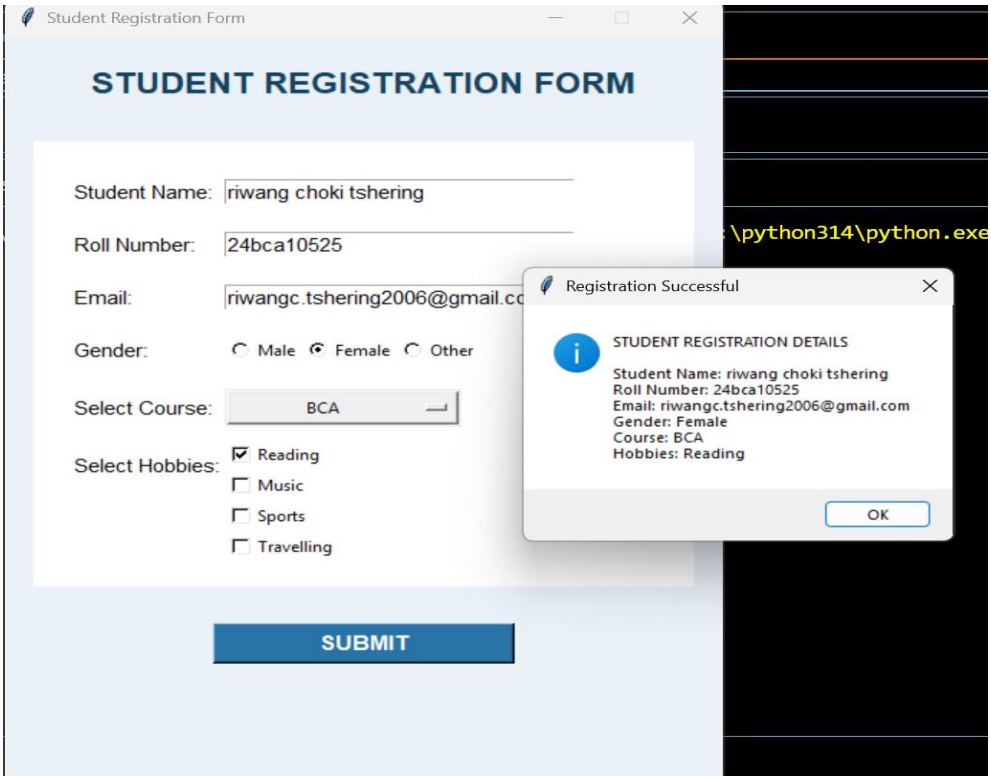
Python Software Foundation. The Python Tutorial. https://docs.python.org/3/tutorial/

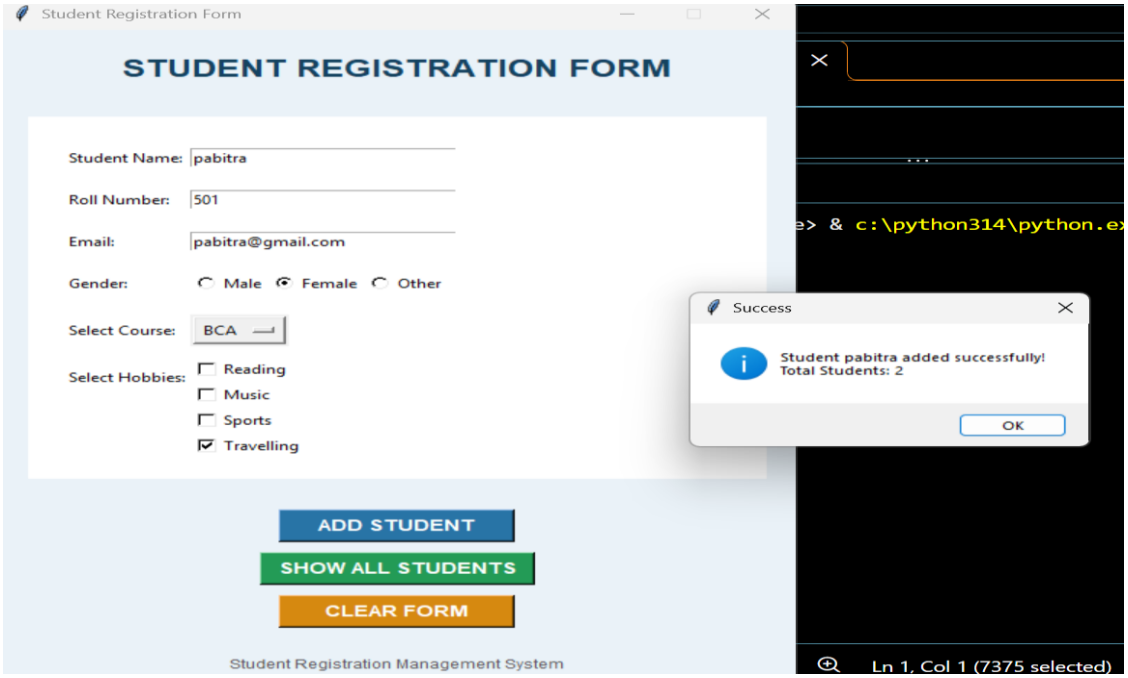
TkDocs. Tkinter Tutorial. https://tkdocs.com/

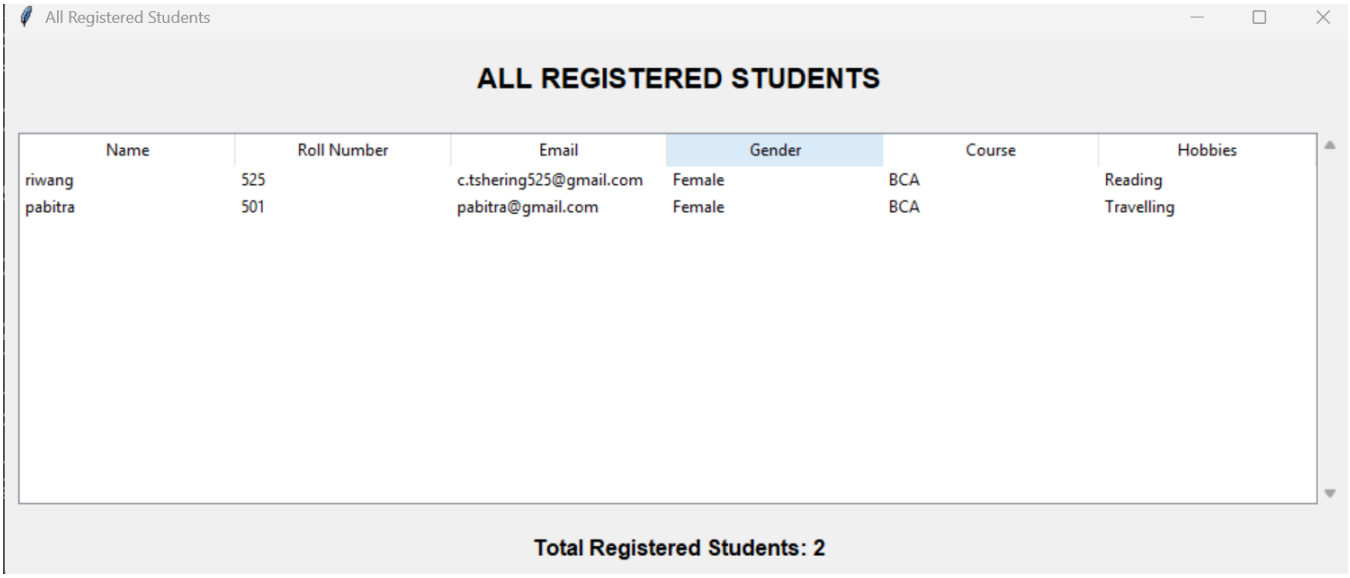
APPENDIX A: KEY TERMS

Term	Definition
GUI	Graphical User Interface: windows, buttons and fields used instead of text commands.
Widget	A visual element of a GUI such as a label, entry box or button.
Event-driven programming	A style in which code runs in response to user actions such as clicks.
Callback function	A function passed to a widget and run when its event occurs.
Validation	Checking that input meets the required rules before it is stored.
Treeview	A ttk widget that shows data in rows and columns.
Dictionary	A Python collection that stores values under named keys.


Student Registration Form
STUDENT REGISTRATION FORM
Student Name:
Roll Number:
Email:
Gender:
Male
Female
Other
Select Course:
BCA
Select Hobbies:
Reading
Music
Sports
Travelling
SUBMIT


Student Registration Form
STUDENT REGISTRATION FORM
Student Name: riwang choki tshering
Roll Number: 24bca10525
Email: riwangc.tshering2006@gmail.cc
Gender:
Male
Female
Other
Select Course:
BCA
Select Hobbies:
Reading
Music
Sports
Travelling
SUBMIT
\python314\python.exe
Registration Successful
STUDENT REGISTRATION DETAILS
Student Name: riwang choki tshering
Roll Number: 24bca10525
Email: riwangc.tshering2006@gmail.com
Gender: Female
Course: BCA
Hobbies: Reading
OK


Student Registration Form
STUDENT REGISTRATION FORM
Student Name: pabitra
Roll Number: 501
Email: pabitra@gmail.com
Gender: Male Female Other
Select Course: BCA
Select Hobbies: Reading Music Sports Travelling
ADD STUDENT
SHOW ALL STUDENTS
CLEAR FORM
Student Registration Management System
Success
Student pabitra added successfully!
Total Students: 2
OK
Ln 1, Col 1 (7375 selected)


All Registered Students
ALL REGISTERED STUDENTS
Name | Roll Number | Email | Gender | Course | Hobbies
riwang | 525 | c.tshering525@gmail.com | Female | BCA | Reading
pabitra | 501 | pabitra@gmail.com | Female | BCA | Travelling
Total Registered Students: 2